Longest Common Subsequence | Introduction & LCS Length

The longest common subsequence (LCS) problem is the problem of finding the longest subsequence that is present in given two sequences in the same order. i.e. find a longest sequence which can be obtained from the first original sequence by deleting some items, and from the second original sequence by deleting other items.

The problem differs from problem of finding common substrings. Unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences.

For example, consider the two following sequences X and Y

```
X: ABCBDAB
Y: BDCABA
```

```
The length of LCS is 4
```

Browse

Adobe Algorithm Amazon BFS Binary Search Bit Hacks Collections DFS FIFO Generics Google Greedy Guava Hashing Intro Java 8 Java 9 JSON Lambda LCS LIFO Maze Memoized Microsoft MIT Must Know Naive Priority Queue Probability Recursive Searching Sliding Window STL Streams Tabulation Tricky Trie

```
LCS are BDAB, BCAB and BCBA
```

A Naive solution would be to check if every subsequence of `X[1..m]` to see if it is also a subsequence of `Y[1..n]`. As there are 2^m subsequences possible of X, the complexity of this solution would be $O(n.2^m)$.

The LCS problem has an optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can be broken down into yet simpler subproblems, and so on, until, finally, the solution becomes trivial.

1. Let us consider two sequences X and Y of length m and n that both end in the same element.

   To find their LCS, shorten each sequence by removing the last element, find the LCS of the shortened sequences, and to that LCS append the removed element. So we can say that

```
LCS(X[1..m], Y[1..n]) = LCS(X[1..m-1],
Y[1..n-1]) + X[m]   if X[m] = Y[n]
```

2. Now suppose that the two sequences do not end in the same symbol.

   Then the LCS of X and Y is the longer of the two sequences `LCS(X[1..m-1], Y[1..n])` and `LCS(X[1..m], Y[1..n-1])`. To understand this property, let's consider the two following sequences

```
X: ABCBDAB (n elements)
```

```
Y: BDCABA  (m elements)
```

The LCS of these two sequences either ends with a B (the last element of sequence X) or does not.

Case 1: If LCS ends with a B, then it cannot end with a A and we can remove the A from sequence Y and the problem reduces to `LCS(X[1..m], Y[1..n-1])`.

Case 2: If LCS does not end with a B, then we can remove B from the sequence X and the problem reduces to `LCS(X[1..m-1], Y[1..n])`. For example,

```
LCS(‘ABCBDAB’, ‘BDCABA’) = maximum
(LCS(‘ABCBDA’, ‘BDCABA’),
                                    L
CS(‘ABCBDAB’, ‘BDCAB’))

LCS(‘ABCBDA’, ‘BDCABA’)
= LCS(‘ABCBD’, ‘BDCAB’) + ‘A’
LCS(‘ABCBDAB’, ‘BDCAB’) =
LCS(‘ABCBDA’, ‘BDCA’) + ‘B’

LCS(‘ABCBD’, ‘BDCAB’) = maximum
(LCS(‘ABCB’, ‘BDCAB’), LCS(‘ABCBD’,
‘BDCA’))
LCS(‘ABCBDA’, ‘BDCA’) = LCS(‘ABCBD’,
‘BDC’) + ‘A’

and so on..
```

Below solution finds the length of LCS of sequences `X[0..m-1]` and `Y[0..n-1]` recursively by using optimal substructure property of LCS problem.

C++

```
#include <bits/stdc++.h>
using namespace std;

// Function to find length of Longest
Common Subsequence of
// sequences X[0..m-1] and Y[0..n-1]
int LCSLength(string X, string Y, int
m, int n)
{
    // return if we have reached the
end of either sequence
    if (m == 0 || n == 0)
        return 0;

    // if last character of X and Y
matches
    if (X[m - 1] == Y[n - 1])
        return LCSLength(X, Y, m - 1, n
- 1) + 1;

    // else if last character of X and
Y don't match
    return max(LCSLength(X, Y, m, n -
1), LCSLength(X, Y, m - 1, n));
}

// Longest Common Subsequence
int main()
{
    string X = "ABCBDAB", Y = "BDCABA";

    cout << "The length of LCS is " <<
            LCSLength(X, Y, X.length(),
Y.length());

    return 0;
}
```

Download Code Run Code

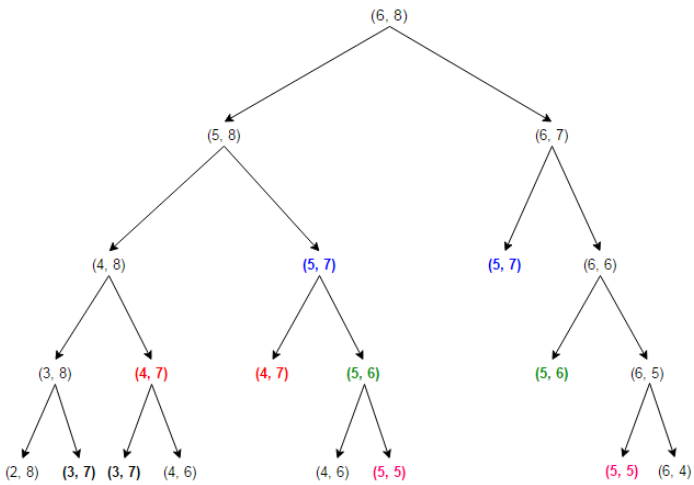
Output:

```
The length of LCS is 4
```

The worst case time complexity of above solution is $O(2^{(m+n)})$. The worst case happens when there is no common subsequence present in X and Y (i.e. LCS is `0`) and each recursive call will end up in two recursive calls.

The LCS problem exhibits overlapping subproblems. A problem is said to have overlapping subproblems if the recursive algorithm for the problem solves the same subproblem over and over rather than always generating new subproblems.

Let us consider recursion tree for two sequences of length 6 and 8 whose LCS is `0`.



As we can see, the same sub-problems (highlighted in same color) are getting computed again and again. We know that problems having optimal substructure and overlapping subproblems can be solved by dynamic programming, in which subproblem solutions are memoized rather than computed again and again. This method is illustrated below –

C++

```
#include <bits/stdc++.h>
using namespace std;

// Function to find length of Longest
Common Subsequence of substring
// X[0..m-1] and Y[0..n-1]
int LCSLength(string X, string Y, int
m, int n, auto &lookup)
{
```

```
    // return if we have reached the end of either string
    if (m == 0 || n == 0)
        return 0;

    // construct a unique map key from dynamic elements of the input
    string key = to_string(m) + "|" + to_string(n);

    // if sub-problem is seen for the first time, solve it and
    // store its result in a map
    if (lookup.find(key) == lookup.end())
    {
        // if last character of X and Y matches
        if (X[m - 1] == Y[n - 1])
            lookup[key] = LCSLength(X, Y, m - 1, n - 1, lookup) + 1;

        else
        // else if last character of X and Y don't match
        lookup[key] = max(LCSLength(X, Y, m, n - 1, lookup),
                          LCSLength(X, Y, m - 1, n, lookup));
    }

    // return the subproblem solution from the map
    return lookup[key];
}

// Longest Common Subsequence
int main()
{
    string X = "ABCBDAB", Y = "BDCABA";

    // create a map to store solutions of subproblems
    unordered_map<string, int> lookup;

    cout << "The length of LCS is "
        << LCSLength(X, Y, X.length(), Y.length(), lookup);

    return 0;
}
```

Download Code Run

Output:

```
The length of LCS is 4
```

The time complexity of above solution is `O(mn)` and auxiliary space used by the program is `O(mn)`. Note that we can also use array instead of map. Check implementation here.

Above ***Memo***ized version follows the top-down approach, since we first break the problem into subproblems and then calculate and store values. We can also solve this problem in bottom-up manner. In the bottom-up approach, we calculate the smaller values of `LCS(i, j)` first, then build larger values from them.

```
            |
0                                           if
i == 0 or j == 0
LCS[i][j] = | LCS[i – 1][j – 1] +
1                       if X[i-1] == Y[j-1]
            | longest(LCS[i – 1][j], LCS[i][j
– 1])      if X[i-1] != Y[j-1]
```

Let X be **"XMJYAUZ"** and Y be **"MZJAWXU"**. The longest common subsequence between X and Y is **"MJAU"**. The table below is generated by the function `LCSLength`, shows the lengths of the longest common subsequences between prefixes of X and Y. The ith row and jth column shows the length of the LCS of substring `X[0..i-1]` and `Y[0..j-1]`.

		0	1	2	3	4	5	6	7
		Ø	M	Z	J	A	W	X	U
0	Ø	0	0	0	0	0	0	0	0
1	X	0	0	0	0	0	0	1	1
2	M	0	1	1	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3	3	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to find length of Longest
Common Subsequence of substring
// X[0..m-1] and Y[0..n-1]
int LCSLength(string X, string Y)
{
    int m = X.length(), n = Y.length();

    // lookup table stores solution to
already computed sub-problems
    // i.e. lookup[i][j] stores the
length of LCS of substring
    // X[0..i-1] and Y[0..j-1]
    int lookup[m + 1][n + 1];

    // first column of the lookup table
will be all 0
    for (int i = 0; i <= m; i++)
        lookup[i][0] = 0;

    // first row of the lookup table
will be all 0
    for (int j = 0; j <= n; j++)
        lookup[0][j] = 0;

    // fill the lookup table in bottom-
up manner
    for (int i = 1; i <= m; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            // if current character of
X and Y matches
            if (X[i - 1] == Y[j - 1])
```

```
                lookup[i][j] = lookup[i - 1][j - 1] + 1;

            // else if current character of X and Y don't match
            else
                lookup[i][j] = max(lookup[i - 1][j], lookup[i][j - 1]);
        }
    }

    // LCS will be last entry in the lookup table
    return lookup[m][n];
}

// Longest Common Subsequence
int main()
{
    string X = "XMJYAUZ", Y = "MZJAWXU";

    cout << "The length of LCS is " << LCSLength(X, Y);

    return 0;
}
```

Download Code Run Code

Output:

```
The length of LCS is 4
```

The time complexity of above solution is `O(mn)` and auxiliary space used by the program is `O(mn)`. The space compexity of above solution can be improved to `O(n)` as calculating LCS of a row of the LCS table requires only the solutions to the current row and the previous row.

Applications of LCS problem:

The longest common subsequence problem forms the

basis of data comparison programs such as the diff utility and use in field of bioinformatics. It is also widely used by revision control systems such as Git.

Exercise:

1. Extend the solution for finding length of LCS for K-sequences

2. Write space optimized code for iterative version.

Recommended Read: Longest Common Subsequence (Finding all LCS)

References:
https://en.wikipedia.org/wiki/Longest_common_subsequence_problem

Thanks for reading.

Email Address Subscribe to Posts

Please use ideone or C++ Shell or any other online compiler link to post code in comments.
Like us? Please spread the word and help us grow.
Happy coding

Sharing is caring:

Tweet Pocket

Dynamic Programming, String

Amazon, LCS, Memoized, Naive, Recursive, STL, Tabulation

Introduction to Dynamic Programming

Longest Common Subsequence (LCS) | Space optimized version

Leave a Reply

Notify of new follow-up comments Email ›

Sort by: newest | oldest | most voted

Geek

Good post!

Guest

1 REPLY

10 months 23 hours ago

Anne

There is a mistake in #2, case 2 of the first solution. Since I can't copy and paste, I'm not going to repeat it, but there is a '+' where it should

Guest

actually be a maximum().

0 REPLY 1 month 4 days ago

Author

Techie Delight

Anne, thanks a lot for bringing this to our notice. We will update the code. Happy coding

0 REPLY

1 month 4 days ago

Techie Delight © 2017 All Rights Reserved. Privacy Policy Contact us